BULACAN POLYTECHNIC COLLEGE

Bachelor of Science and Information System

CT-MAD 213 MOBILE APPLICATION DEVELOPMENT

FINAL PROJECT DOCUMENTATION



ZELORA

An E-Commerce Mobile Application for Customers, Sellers, Delivery Riders and Administrators

Category: Business Applications (E-Commerce Application) | Complexity: Intermediate

Submitted by:

Cabuz, Jewell Ace Jhay V.

Espino, Jerome M.

Eva, Angel T.

Isidro, Kristina Cassandra P.

Pascual, Leira C.

Submitted to:

KYLE JOSEPH G. FABIAN

10/24/2026

1. PROJECT OVERVIEW

1.1 Project Description

Zelora is an e-commerce mobile application developed using Java that provides users with a convenient platform for online shopping. The application allows buyers to search and browse products, view product information, add items to their shopping cart, apply vouchers, proceed to checkout, make payments, and monitor their orders.

The application connects different users involved in the e-commerce process, including buyers, sellers, administrators, and delivery riders. Buyers can browse and purchase products, sellers can process and manage orders, administrators can manage user accounts and system-related concerns, and delivery riders can handle delivery requests and shipments. Zelora also supports important e-commerce processes such as order processing, order cancellation, payment processing, inventory management, delivery management, and database management. The application aims to provide a convenient, organized, and user-friendly shopping experience through a mobile platform.

1.2 Problem Statement

Traditional shopping requires customers to physically visit stores, search for products, and complete purchases in person. This can be time-consuming and inconvenient, especially for customers who prefer using mobile devices for shopping. Sellers may also experience difficulties in managing customer orders, monitoring sales, handling payments, and coordinating deliveries when these processes are performed manually or through separate platforms.

The following problems are addressed by the system:

1. Difficulty in searching and browsing products.
2. Time-consuming traditional shopping processes.
3. Difficulty in managing shopping carts and orders.
4. Lack of a convenient mobile-based ordering platform.
5. Difficulty in monitoring order status.
6. Difficulty in cancelling orders when necessary.
7. Challenges in coordinating deliveries.
8. Difficulty in managing users and seller activities.
9. Lack of an integrated platform connecting buyers, sellers, administrators, and delivery riders.

To address these concerns, Zelora provides an integrated e-commerce mobile application that combines shopping, ordering, payment, cancellation, and delivery-related processes in one platform.

Target Users

Buyers/Customers	Buyers are the primary users of Zelora. They can browse products, view product information, add products to their shopping cart, apply vouchers, checkout, submit payments, monitor their orders, cancel eligible orders, and rate or review products.
Sellers	Sellers use the application to receive and process orders, ship products, monitor order status, view sales analytics, and manage seller-related activities.
Administrators	Administrators manage user accounts, handle disputes, and monitor important activities within the system.
Delivery Riders	Delivery riders handle delivery-related activities. They can receive delivery requests, accept or reject delivery jobs, confirm pickup, and mark orders as delivered.

1.3 Proposed Solution

The proposed solution is Zelora, a Java-based e-commerce mobile application that provides an integrated platform for online shopping. The application allows buyers to search and browse products, view product information, add products to their shopping cart, apply vouchers, proceed to checkout, make payments, monitor orders, and cancel eligible orders.

The system also provides functions for sellers, administrators, and delivery riders. Sellers can process and ship orders, administrators can manage users and system concerns, while delivery riders can receive and complete delivery requests. Through these functions, Zelora aims to make the online shopping process more convenient, organized, and accessible.

2. OBJECTIVES, SCOPE AND FEATURES

2.1 General Objective

The general objective of the project is to develop a functional and user-friendly Java-based e-commerce mobile application called Zelora that integrates product browsing, shopping cart management, checkout, payment, order cancellation, order processing, and delivery management into one platform.

2.2 Specific Objectives

1. Develop a functional e-commerce mobile application using Java.
2. Provide users with an easy-to-use mobile shopping platform.
3. Allow buyers to search and browse available products.
4. Allow buyers to view detailed product information.
5. Allow buyers to add products to their shopping cart.
6. Allow buyers to apply available vouchers.
7. Provide a convenient checkout process.
8. Support simulated payment processing (Card, GCash/Maya and Cash on Delivery).
9. Allow buyers to monitor their order status.
10. Allow buyers to cancel eligible orders.
11. Allow buyers to rate and review purchased products.
12. Allow sellers to receive and process customer orders.
13. Allow sellers to ship orders and monitor sales.
14. Allow administrators to manage user accounts and system concerns.
15. Allow delivery riders to receive and manage delivery requests.
16. Store and manage important information through a database.
17. Provide an organized and convenient e-commerce experience.

2.3 Scope and Limitations

Scope.

Zelora covers the major processes involved in an e-commerce system.

Buyer Functions

The buyer can:

- Register an account.
- Log in to the application.
- Search and browse products.
- View product information.
- Add products to the shopping cart.
- Continue shopping.

- View and manage the cart.
- Proceed to checkout.
- Apply available vouchers.
- Initiate payment.
- Submit payment.
- Monitor order status.
- Cancel eligible orders.
- Receive order notifications.
- Rate and review products.

Seller Functions

The seller can:

- Receive order notifications.
- View customer orders.
- Process customer orders.
- Ship orders.
- Monitor order status.
- View sales analytics.
- Manage seller-related activities.

Admin Functions

The administrator can:

- Manage user accounts.
- Handle disputes.
- Monitor system activities.
- Manage delivery-related requests.

Delivery Rider Functions

The delivery rider can:

- Receive delivery requests.
- Accept or reject delivery jobs.
- Confirm pickup.
- Mark orders as delivered.
-

System Functions

The system includes:

- User authentication.
- Product browsing.
- Shopping cart management.
- Order processing.

- Order cancellation.
- Simulated payment processing.
- Inventory control.
- Database management.

Limitations

The application may have the following limitations:

- Data is stored only on the phone, so accounts and orders are not shared between devices.
- Payments are simulated; no real payment gateway or money transfer is used.
- The map on the Delivery Details screen is a placeholder; there is no real-time rider tracking.
- Route navigation and optimization are not included in the current version.
- Product availability depends on information stored in the database.
- Order cancellation may only be available while the order is still eligible for cancellation.
- Some advanced e-commerce features may not be available in the current version.
- The application is primarily designed for Android mobile devices.
- System performance may depend on the user's device.

2.4 Major Features

Role	Major features
All users	Sign up with role choice, login, forgot password (offline reset), persistent session, log out, account settings, Help Center and Terms and Conditions.
Customer	Home with search, category chips and sort; product detail with photo slider, colors, quantity, wishlist and share; wishlist; cart; checkout sheet with address, payment method (Card, GCash/Maya, Cash on Delivery) and voucher codes; My Orders with cancel while Processing, rate and review, report a problem; order status notifications.
Seller	Receive order notifications, view and process customer orders, ship orders, monitor order status, view sales analytics, and manage products and the shop profile.
Rider	Receive delivery requests, go online, accept or reject delivery jobs (the first rider to accept takes the order), confirm pickup, mark orders as delivered, and view earnings in the wallet.
Admin	Manage user accounts, handle disputes, monitor system activities, and manage delivery-related requests.

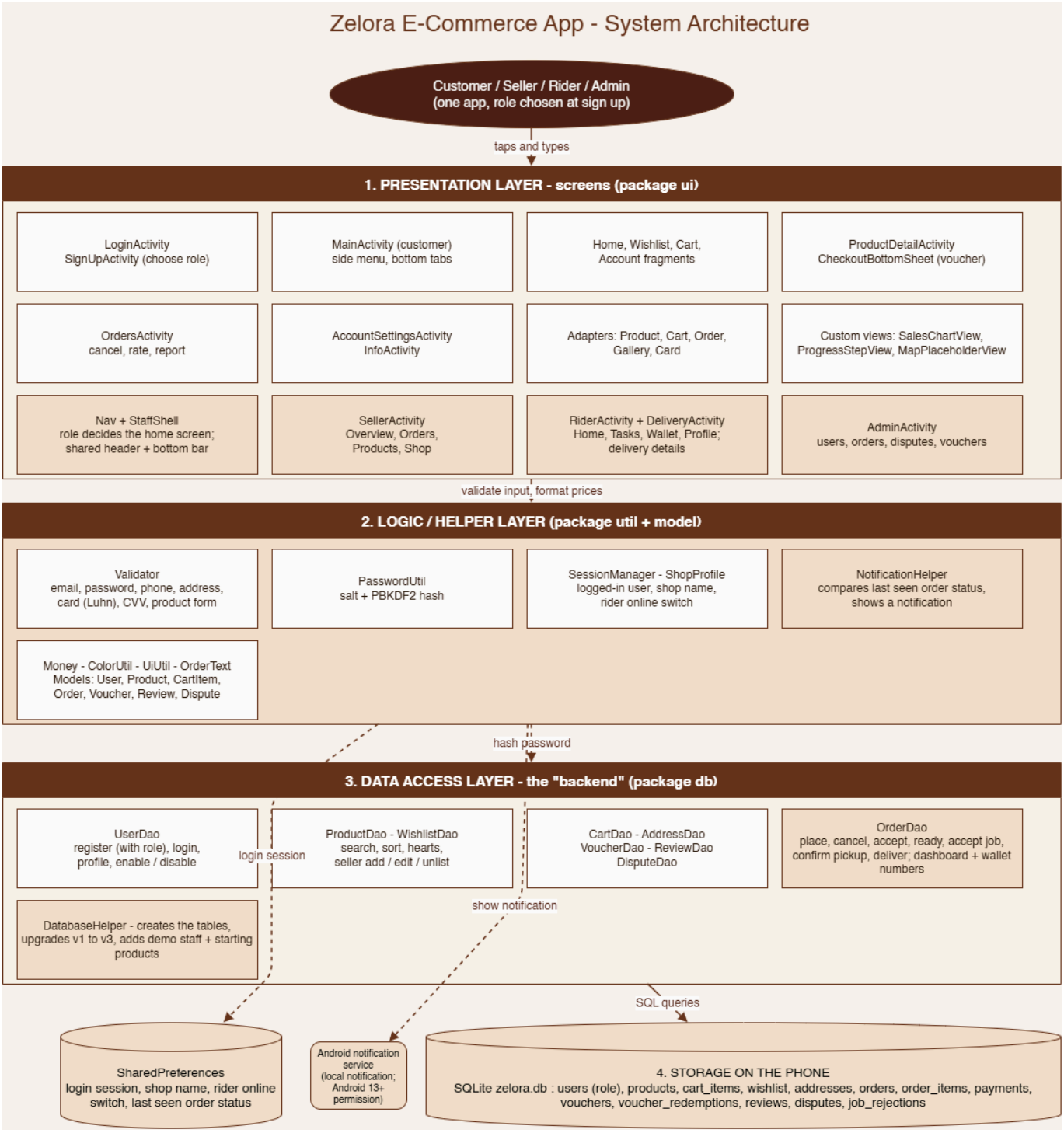
3. SYSTEM DESIGN

This section shows how Zelora is built. The architecture diagram shows the layers of the app, the process flow shows how a customer order moves through the system and the database, the order lifecycle shows which role moves an order at each step, the use case diagram shows what each role can do, and the ER diagram shows the SQLite database. The source files of all diagrams (Graphviz and draw.io) are in the project's docs folder on GitHub.

3.1 System Architecture

The app has four layers. The presentation layer holds the screens (activities, fragments and adapters). The logic layer holds validation, password hashing, session handling and the notification helper. The data access layer (DAO classes and DatabaseHelper) is the backend: every SQL query is written there with parameters. The data is stored in an SQLite database on the phone, and small settings such as the logged-in user are kept in Shared Preference.

System Architecture Diagram



3.2 Application Process Flow

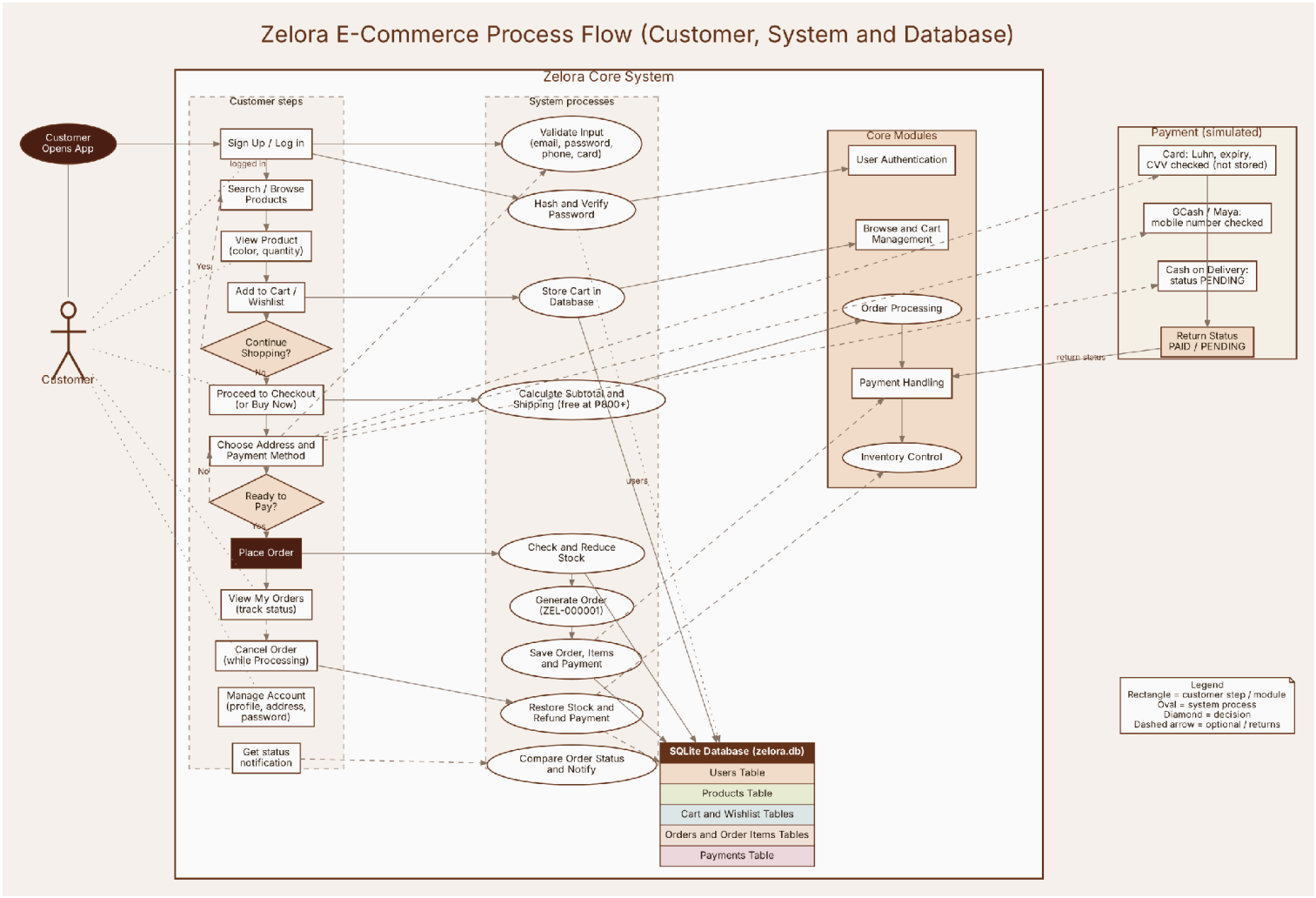


Figure 2. Process flow of the customer, the system processes and the SQLite database.

3.3 Order Lifecycle

An order is created as Processing. The customer can cancel only while it is Processing. The seller then accepts it (Preparing) and marks it ready (Packed). The first rider to accept the job takes it (Out for Delivery), confirms pickup, and marks it Delivered, at which point a Cash on Delivery payment becomes PAID. A customer can report a problem on a Packed, Out for Delivery or Delivered order, and the admin resolves, refunds or rejects the report.

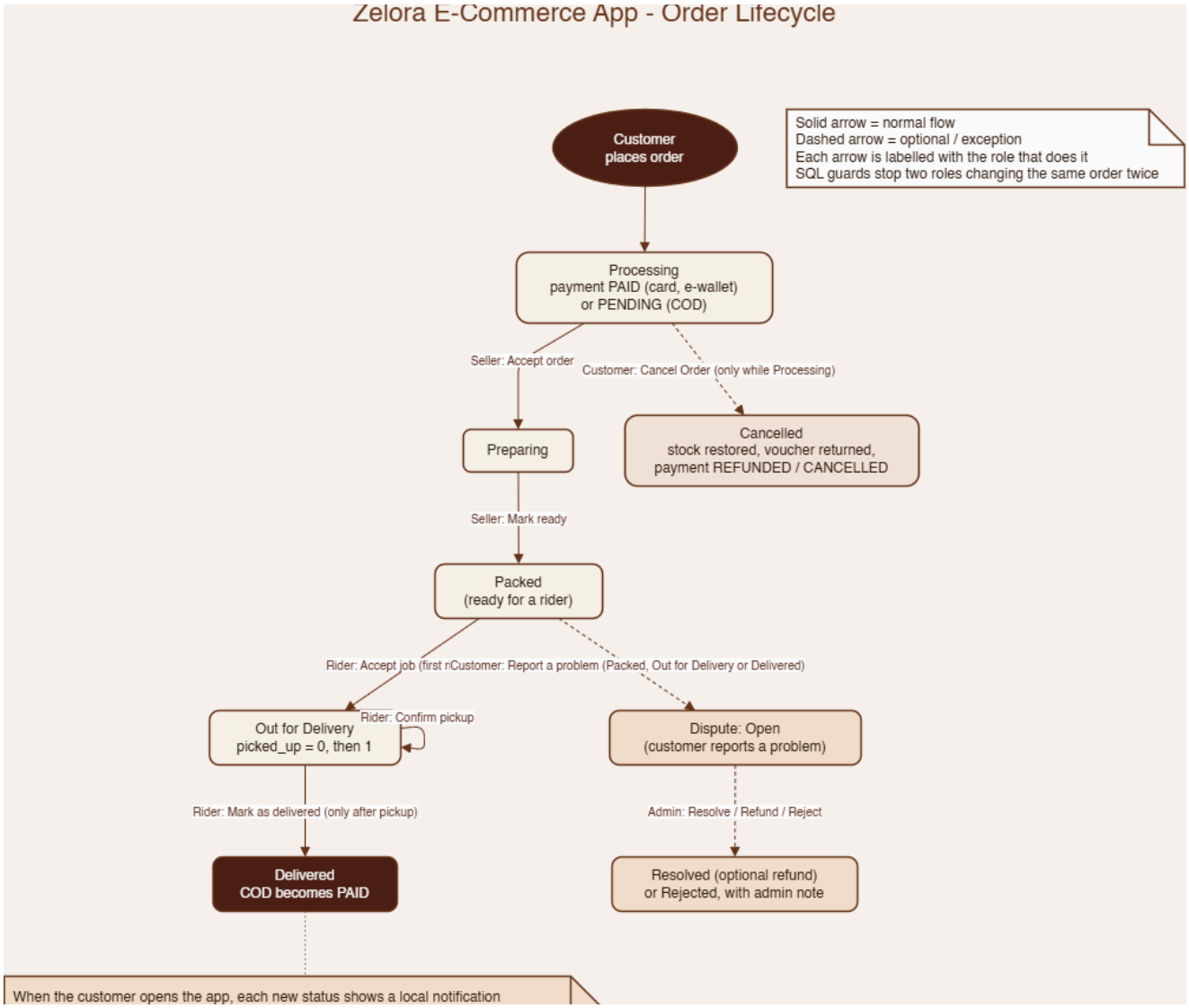


Figure 3. Order lifecycle: which role moves an order at each step.

3.4 Use Case Diagram (Seller, Delivery Rider and Admin)

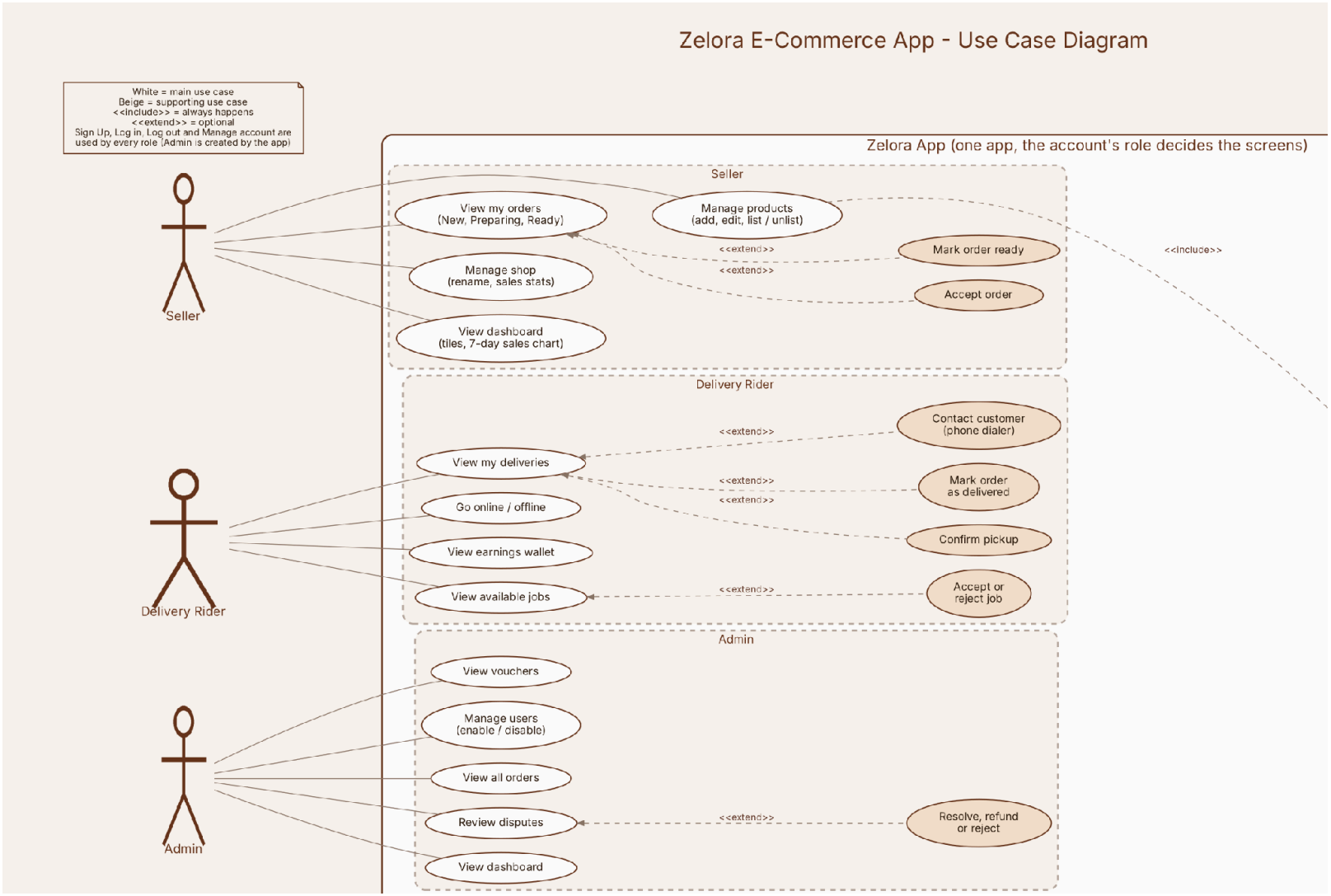


Figure 4. Use cases of the Seller, Delivery Rider and Admin.

Use Case Diagram (Customer and shared account use cases)

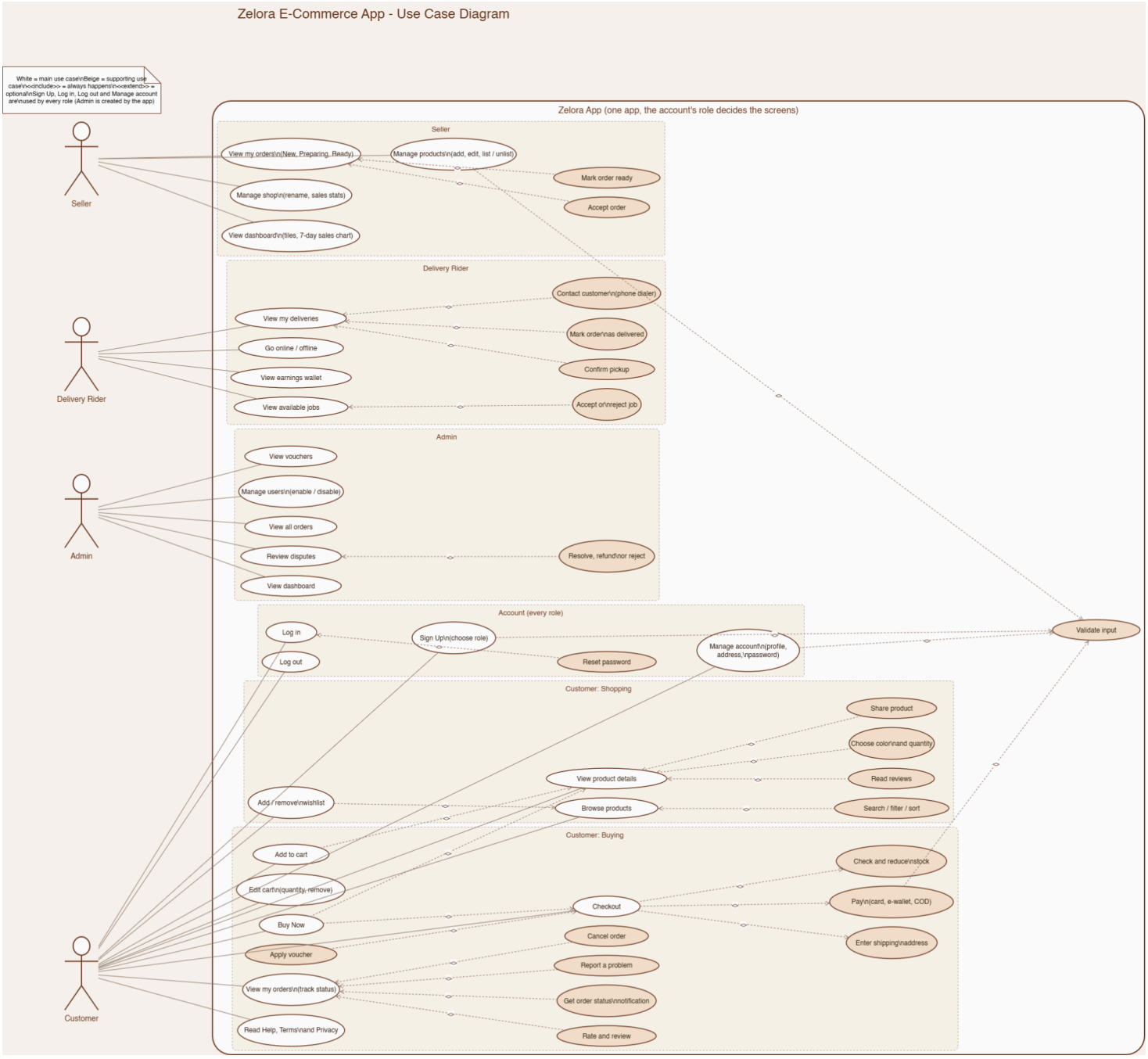


Figure 5. Use cases of the Customer and the account functions used by every role.

3.5 Entity Relationship (ER) Diagram

The database has 13 tables. users hold all four roles in the role column. products belong to a seller. orders belong to a customer and, once accepted, to a rider. order_items store the product name and price at the time of purchase, so later price changes do not change old orders.

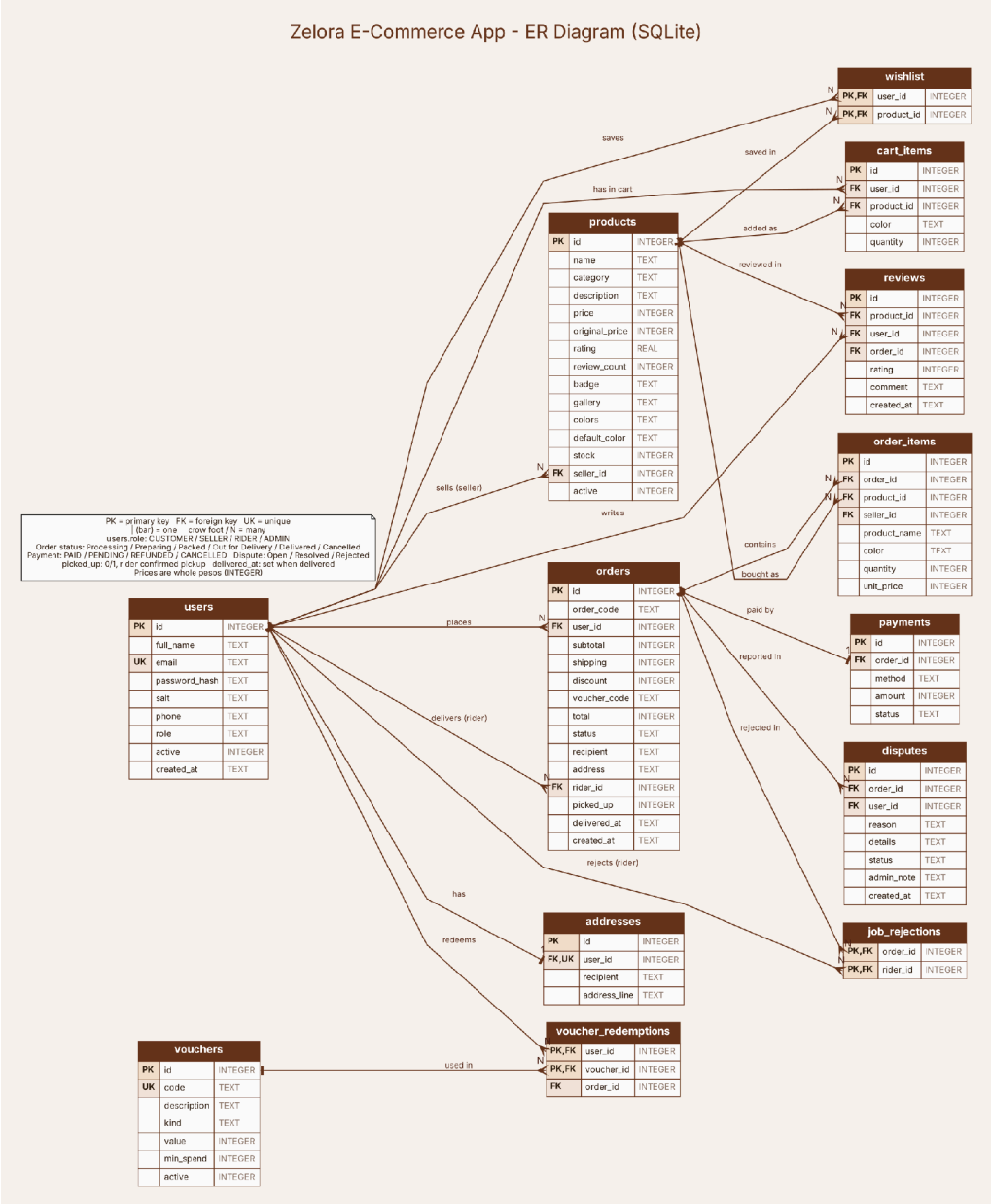


Figure 6. ER diagram of the Zelora SQLite database (zelora.db, version 3).

4. UI/UX AND IMPLEMENTATION

4.1 UI/UX Design

The UI/UX design of Zelora focuses on creating a user-friendly online shopping experience with a minimalist interface featuring warm cream backgrounds and dark brown accents. Utilizing Figma for mockups, the application organizes roles into Customer, Seller, Rider, and Admin, each with tailored interfaces. Customers can browse products and manage orders, Sellers monitor sales and manage listings, Riders handle deliveries, and Admins oversee user accounts and platform statistics. The design prioritizes consistency, readability, and ease of navigation, ensuring efficient task management for all users.

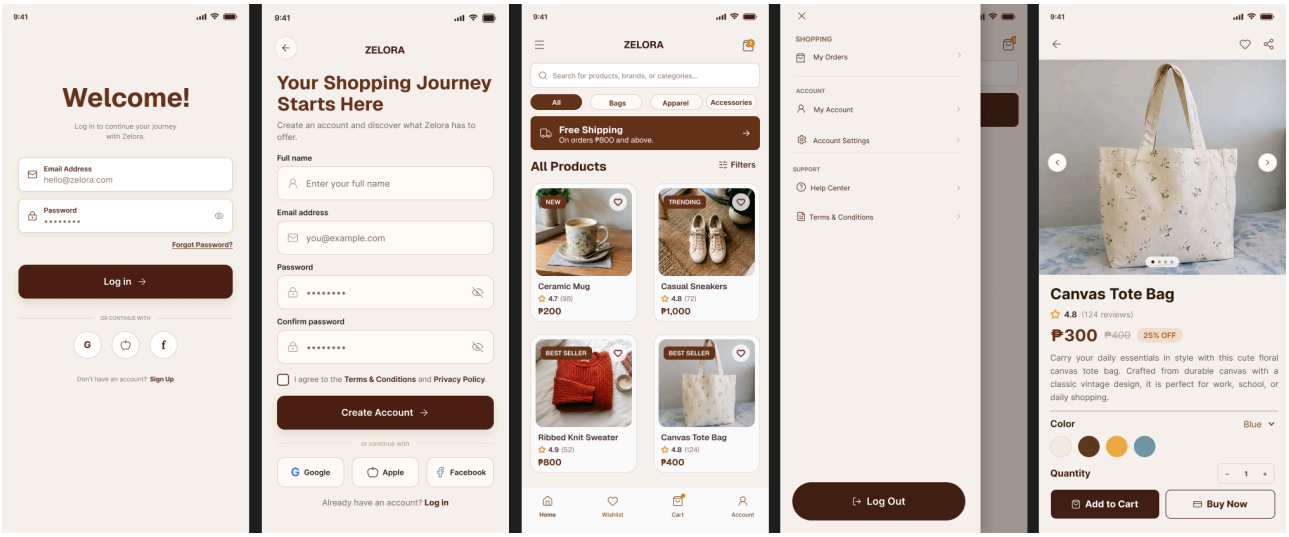


Figure 7. Mockups: Login, Sign Up, Home, Side menu and Product Detail.

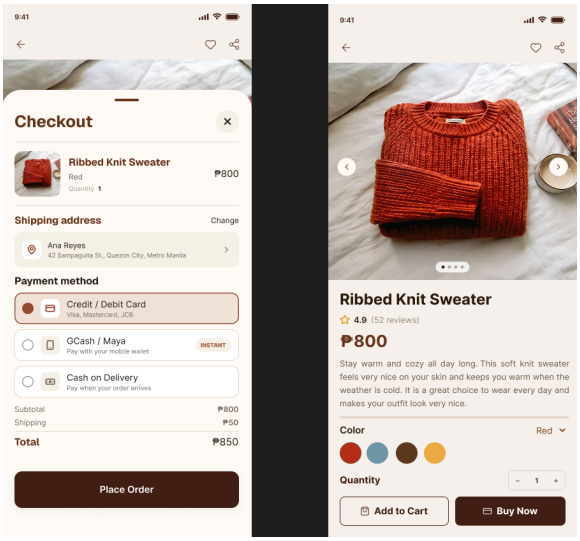


Figure 8. Mockups: Checkout sheet and Product Detail.

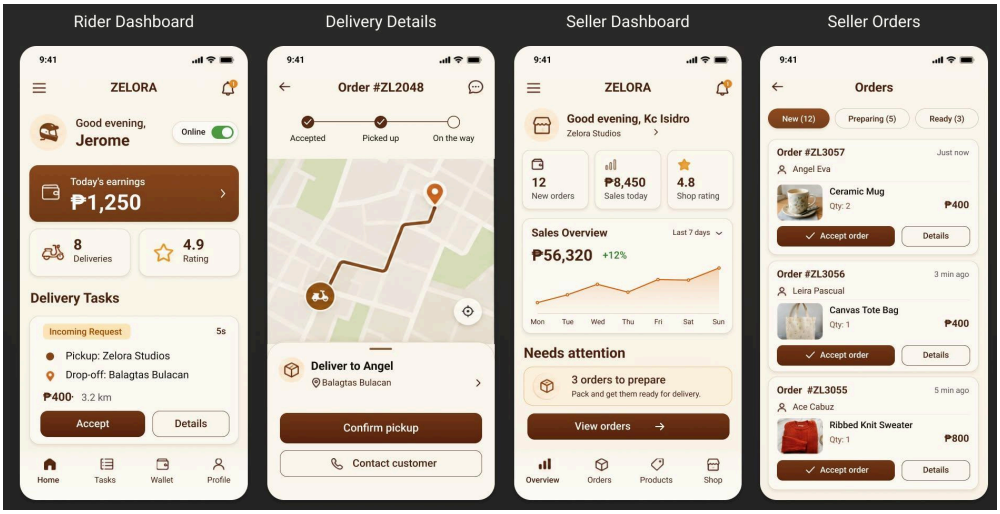


Figure 9. Mockups: rider's dashboard, delivery details and seller's dashboard and sellers orders

Color palette. The colors were taken from the final mockups and are defined once in colors.xml, with names that describe their role.

Color	Name	Hex	Used for
	Cream	#F8F3EB	App background
	Soft white	#FDFBFA	Cards, bottom navigation
	Brand brown	#67331B	Headings, banner, selected chip, prices
	Dark brown	#4B2215	Primary buttons (Log in, Create Account)
	Text brown	#572A1B	Labels and icons
	Muted brown	#8B7468	Subtitles and section labels
	Beige	#F0DFC8	Highlights, table headers, supporting items
	Amber	#E4A853	Rating stars
	Orange	#D78A2C	Cart badge and notification dot
	Selected brown	#9A4E32	Selected payment option

Typography. Headings, titles and prices use Bricolage Grotesque (bold and semibold). Body text, labels and buttons use Inter (regular, medium, semibold and bold). Both are open-source fonts under the SIL Open Font License and are bundled in the app. Text sizes run from about 12 sp for helper text to 26 sp for screen titles.

Navigation structure. Each role has a short, flat navigation with at most four main destinations:

Role	Navigation
Customer	Bottom navigation: Home, Wishlist, Cart, Account. A side menu gives My Orders, My Account, Account Settings, Help Center, Terms and Conditions, and Log Out. Product Detail and the Checkout sheet open from Home and Cart.
Seller	Bottom navigation: Overview, Orders, Products, Shop. A top menu has Log out and a bell that shows how many orders are waiting.
Rider	Bottom navigation: Home, Tasks, Wallet, Profile. Delivery Details opens from a task card.
Admin	One screen with five tabs: Overview, Users, Orders, Disputes, Vouchers.

UI/UX decisions. The decisions below follow the required UI/UX principles:

Principle	How Zelora applies it
Consistency	The same colors, fonts, rounded buttons and card style are used in all four role apps. The seller and rider apps share one frame (header, page and bottom bar).
Usability	Short, clear labels such as Accept order, Mark ready, Confirm pickup and Place Order. Confirmation dialogs appear before steps that cannot be undone. Empty lists show a helpful message.
Visual hierarchy	Large bold titles and prices, small gray helper text, and one dark primary button per screen.
Accessibility	Dark brown text on cream gives strong contrast. Buttons and chips are tall enough to tap comfortably. Icons are paired with text labels.
Responsive layout	Layouts use match_parent, weights and scrolling, so they adapt to different phone sizes. Long screens scroll.
Feedback	Toast messages and field errors tell the user what happened. Notifications report order status changes.

4.2 Development Tools

Tool	Purpose
Android Studio	The primary Integrated Development Environment (IDE) used for code editing, emulator management, Logcat debugging, and building the application.
Gradle 8.9 with Android Gradle Plugin 8.7.3	Builds the project and manages libraries.

Tool	Purpose
Android Emulator / physical phone	Used to run and test the application. A physical Android device can also be connected through USB debugging.
Git and GitHub	Used for version control, source code management, tracking changes, and maintaining the project's repository.
draw.io and Graphviz	Used to create the system diagrams stored in the Zelora-Diagrams.drawio file.
Figma	Used to create UI/UX mockups and plan the application's screen layouts and navigation before implementation.

4.3 Technologies Used

Technology	Use in Zelora
Java 17	Programming language of the application.
Android SDK (minSdk 26, targetSdk 34, compileSdk 35)	Platform APIs: activities, fragments, notifications, intents.
XML layouts	Screen designs and resources.
SQLite (SQLiteOpenHelper)	Local database with 13 tables, DAO classes and parameterized queries.
SharedPreferences	Logged-in user, shop name, rider online switch and last seen order status.
AndroidX AppCompat 1.7.0, RecyclerView 1.3.2, ViewPager2 1.1.0, Material Components 1.12.0	UI components: lists, photo slider, dialogs, bottom navigation.
PBKDF2 with salt	Password hashing (100,000 iterations).
NotificationManager and NotificationChannel	Local order status notifications.

4.4 Major Application Screens

Zelora contains more than 20 screens organized into four user roles: Customer, Seller, Rider, and Admin. Each role has a dedicated interface designed to support its specific responsibilities. The application also includes shared Login and Sign Up screens. After logging in, users are directed to the home screen corresponding to their assigned role.

Screen	Purpose
Login screen	The Login screen allows registered users to access the application using their email address and password. It includes the Zelora logo, login fields, a login button, and a forgot-password option. Disabled accounts are refused access..
Sign Up Screen	The Sign Up screen allows new users to create an account and select an available role, such as Customer, Seller, or Rider. The form validates user input before registration. The selected role determines the interface and functions available after login.
Customer Screens	The Customer interface supports the complete online shopping process. Its bottom navigation bar provides access to Home, Wishlist, Cart, and Account.
Home Screen	The Home screen displays the product catalog and provides product search and category filters. Customers can browse available products and view their images, names, ratings, and prices.
Product Detail Screen	The Product Detail screen displays the selected product's photo gallery, description, available colors, and customer reviews. Customers can add products to their cart or wishlist and proceed directly to purchasing.
Wishlist Screen	The Wishlist screen displays products saved by customers for future reference, allowing them to keep track of items they may want to purchase later.
Cart Screen	The Cart screen displays the products selected for purchase. Customers can change quantities, remove items, and apply available vouchers before proceeding to checkout.
Checkout Screen	The Checkout screen presents the customer's shipping address, available payment methods, and order summary. It includes Cash on Delivery as a payment option and displays the relevant order details before the customer places an order.
My Orders Screen	The My Orders screen lists the customer's orders with a status tracker (Processing, Preparing, Packed, Out for Delivery, Delivered). The customer can cancel an order while it is processing, rate and review delivered items, and report a problem on an order that is Packed, Out for Delivery or Delivered.
Seller Screens	The Seller interface has four tabs: Overview (sales chart and order counts), Orders (Accept order, then Mark ready), Products (add and manage products and stock) and Shop (shop name and profile).

Screen	Purpose
Rider Screens	The Rider interface has Home (online switch and available delivery jobs), Tasks (the rider's jobs), Wallet (earnings of ₱40 per delivery) and Profile. Delivery Details shows the progress (Accepted, Picked up, Delivered) with the buttons Confirm pickup and Mark as delivered.
Admin Screens	The Admin interface has five tabs: Overview, Users (enable or disable accounts), Orders, Disputes (resolve, refund or reject reports) and Vouchers.

5. TESTING AND EVALUATION

5.1 Test Cases

The test cases below cover the required functions of the app. Run each case on the emulator or a phone, then replace the highlighted Result cell with Pass or Fail (and a short note if it failed).

No.	Test case	Steps / input	Expected result	Result
T1	Sign up with valid data	Enter name, email, strong password, matching confirmation, tick Terms; tap Create Account	Account is created and the user is logged in	[Pass / Fail]
T2	Sign up with invalid data	Wrong email format; password too short; passwords do not match	A clear error is shown on the field; no account is created	[Pass / Fail]
T3	Sign up with an email already used	Register the same email twice	Error message; no duplicate account	[Pass / Fail]
T4	Login with correct details	Enter the registered email and password	The home screen of the user's role opens	[Pass / Fail]
T5	Login with a wrong password	Enter a wrong password	"Incorrect email or password" is shown	[Pass / Fail]
T6	Log out and log in again	Log out, then log in with the same account	Login succeeds	[Pass / Fail]
T7	Search and category filter	Type "mug"; tap the Bags chip	Only matching products are listed	[Pass / Fail]
T8	Add to cart	Open a product, choose color and quantity, tap Add to Cart	Item appears in the cart; cart badge shows	[Pass / Fail]
T9	Quantity above stock	Add more units than are in stock and place the order	Out of stock message; nothing is saved	[Pass / Fail]
T10	Checkout without an address	Open Checkout and leave the address empty	Field error; the order is not placed	[Pass / Fail]

No.	Test case	Steps / input	Expected result	Result
T11	Invalid card number	Choose Card and enter a number that fails the Luhn check	Field error is shown	[Pass / Fail]
T12	Place an order (Cash on Delivery)	Complete checkout with Cash on Delivery	Order code appears; order is Processing; payment is PENDING; stock is reduced; cart is emptied	[Pass / Fail]
T13	Apply a voucher	Enter WELCOME10 at checkout	10% discount is applied; the same voucher cannot be used twice by the same customer	[Pass / Fail]
T14	Cancel a Processing order	My Orders → Cancel Order	Status is Cancelled; stock is restored; payment is CANCELLED or REFUNDED	[Pass / Fail]
T15	Cancel after the seller accepted	Open an order that is Preparing or later	No Cancel button is shown	[Pass / Fail]
T16	Seller adds a product with bad data	Empty name, price 0 or negative stock	Field errors; product is not saved	[Pass / Fail]
T17	Seller handles an order	Accept order, then Mark ready	Status changes to Preparing, then Packed	[Pass / Fail]
T18	Rider delivers an order	Go Online, Accept job, Confirm pickup, Mark as delivered	Status becomes Delivered; Cash on Delivery payment becomes PAID; earnings increase	[Pass / Fail]
T19	Deliver before pickup	Try to mark delivered before Confirm pickup	Not allowed; the Mark as delivered button is not offered	[Pass / Fail]
T20	Disabled account	Admin disables a user; the user tries to log in	Login is refused	[Pass / Fail]
T21	Admin resolves a dispute	Customer reports a problem; admin chooses Refund	Dispute is Resolved and the payment is REFUNDED	[Pass / Fail]
T22	Order status notification	Change an order's status as seller or rider, then open the app as the customer	A notification shows the new status	[Pass / Fail]

No.	Test case	Steps / input	Expected result	Result
T23	Notification permission denied	Deny the notification permission on Android 13+	The app does not crash	[Pass / Fail]
T24	Upgrade from an older version	Install over an older version of the app	Existing accounts and orders are kept	[Pass / Fail]

5.2 Testing Results

Checks performed during development. The complete Java source was compiled against the Android API 34 library with no errors. The database layer was tested with SQL scripts: all 13 tables are created with their keys, an upgrade from the first database version keeps its data and adds the new columns, a delivered Cash on Delivery order becomes PAID, and a customer cannot review the same order item twice. Layout files were checked to be valid XML, and every view id used in the Java code was checked against the layouts. These checks do not replace running the app, so the table above must be completed on a device.

Results on the device. A total of 24 test cases were conducted on an Android emulator (Pixel, API 34). All 24 test cases passed, and no test cases failed. The tests covered user registration and login, product searching and filtering, cart management, checkout, payment processing, voucher application, order cancellation, seller and rider functions, notifications, account security, dispute resolution, and app upgrading. The results showed that the application performed as expected during testing, with its main features functioning properly. No major errors were encountered during the testing process. In addition, the testing confirmed that the different functions worked correctly based on their expected results. The successful test execution indicated that the application was stable and that its main features were ready for further evaluation and improvement.

6.CONCLUSION AND RECOMMENDATIONS

Project Accomplishments

The Zelora E-Commerce Mobile Application effectively integrates online shopping, product management, delivery, and administrative functions within a single Android platform. Developed in Java 17 using Android Studio, it features four user roles: Customer, Seller, Rider, and Admin. Customer functionalities include product browsing, order tracking, and reviews, while seller capabilities cover product and order management. Delivery functionalities enable accepting jobs, confirming pickup and delivery, and managing earnings. The admin interface aids in account and order monitoring. Key technologies used include SQLite for local storage and PBKDF2 for password security, with role-based access and workflow restrictions ensuring data protection. This project provided valuable insights into Java programming, Android development, and application security.

Problems Encountered

During the development of the Zelora E-Commerce Mobile Application, the developers encountered several technical challenges involving database persistence, registration, application logic, build configuration, and code syntax.

Database and Account Registration Errors

One of the main problems encountered was that registered accounts and other data were not consistently saved or retained, particularly when the application depended on a connected development environment. Database-related errors also occurred during registration, causing some account creation attempts to fail. These problems made it difficult to ensure consistent account storage and reliable registration.

Logical Errors in the Checkout Process

Another problem involved the order status during checkout. In some cases, an order was automatically marked as Delivered immediately after checkout, even though the delivery process had not yet taken place. This resulted in an incorrect order status and affected the expected sequence of order processing and delivery.

Gradle Build Errors and Hardware Limitations

The project also encountered numerous Gradle configuration and build errors. These issues were more challenging to troubleshoot because the development laptop had limited hardware resources. The laptop's performance limitations affected the development workflow and made building, debugging, and resolving project configuration issues more difficult.

Syntax Errors and Comment Formatting Issues

Syntax errors also occurred during coding, particularly when comments became mixed with code or were incorrectly formatted. These mistakes sometimes caused compilation errors or made the source code more difficult to read and maintain. Identifying and correcting these errors required additional code inspection and debugging.

These challenges highlighted the importance of reliable data persistence, correct application logic, proper build configuration, sufficient development resources, and careful code organization when developing an Android application.

Recommendations and Future Improvements

Based on the problems encountered during development, the following improvements are recommended for future versions of Zelora.

Improve Database Persistence and Registration Reliability

The database implementation should be reviewed to ensure that registered accounts and other important records are saved consistently and remain available after the application is closed or reopened. Registration validation and database operations should also be improved to prevent failed account creation and provide clear error messages when a database operation is unsuccessful. If the application is intended to support account access across multiple devices, a properly configured remote database or backend service may be considered.

Correct the Checkout and Order Status Workflow

The checkout process should be improved to ensure that placing an order does not automatically mark it as Delivered. Order statuses should follow the intended workflow, such as Processing, Preparing, Packed, and Delivered, with each transition occurring only after the appropriate action has been completed. Testing should cover checkout, order acceptance, preparation, pickup, and delivery to ensure that order records remain consistent throughout the process.

Optimize Gradle Configuration and Development Performance

The Gradle configuration and project dependencies should be reviewed to resolve build errors and improve build reliability. Unnecessary dependencies and background processes should be minimized where possible. Future development may also benefit from using a laptop with better hardware specifications to improve build times, emulator performance, and debugging efficiency.

Improve Code Quality and Error Prevention

The source code should be organized using consistent naming conventions, indentation, and comment formatting. Comments should be separated properly from executable code, and the application should be compiled regularly to detect syntax errors early. Code reviews, debugging tools, and automated tests can also help identify errors before they affect application functionality.

Strengthen Security and Data Validation

Future versions should continue improving authentication, role-based access controls, input validation, and protection of user information. Database operations and sensitive functions should verify that users are authorized to perform the requested actions. Error handling should also avoid exposing sensitive information.

Enhance Delivery and Mapping Features

The rider's map placeholder may be replaced with an integrated mapping service that supports real location information and navigation, subject to available resources and permissions. The delivery workflow should continue to enforce pickup confirmation before completion and prevent multiple riders from claiming the same delivery.

Improve Rider Earnings Management

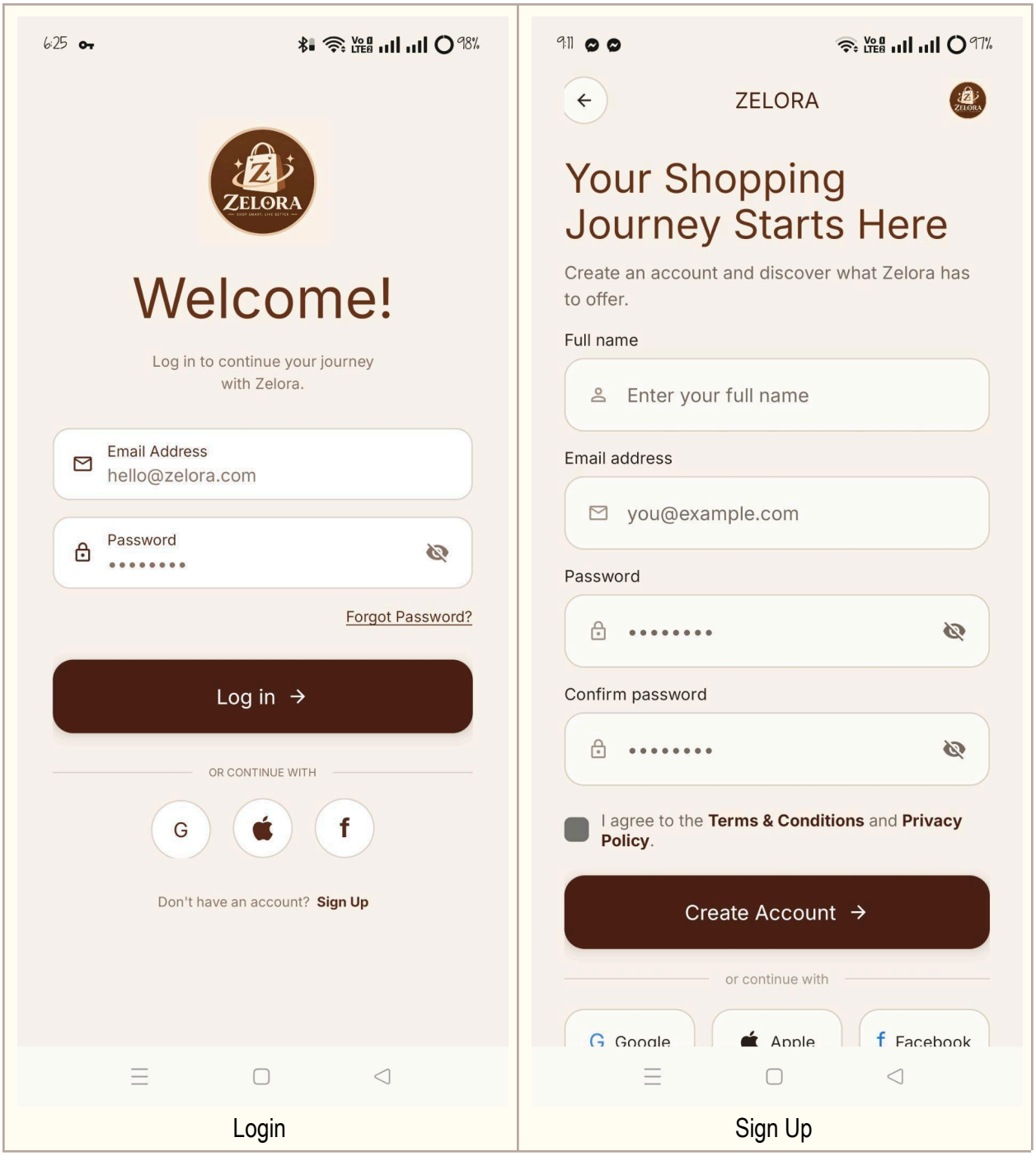
The current flat rate of ₱40 per delivery is an application-defined value and does not represent actual payroll. Future improvements could introduce configurable delivery rates, more detailed earnings reports, and a clearly defined payment calculation system if the application is expanded for real-world operations.

Conduct Comprehensive Testing

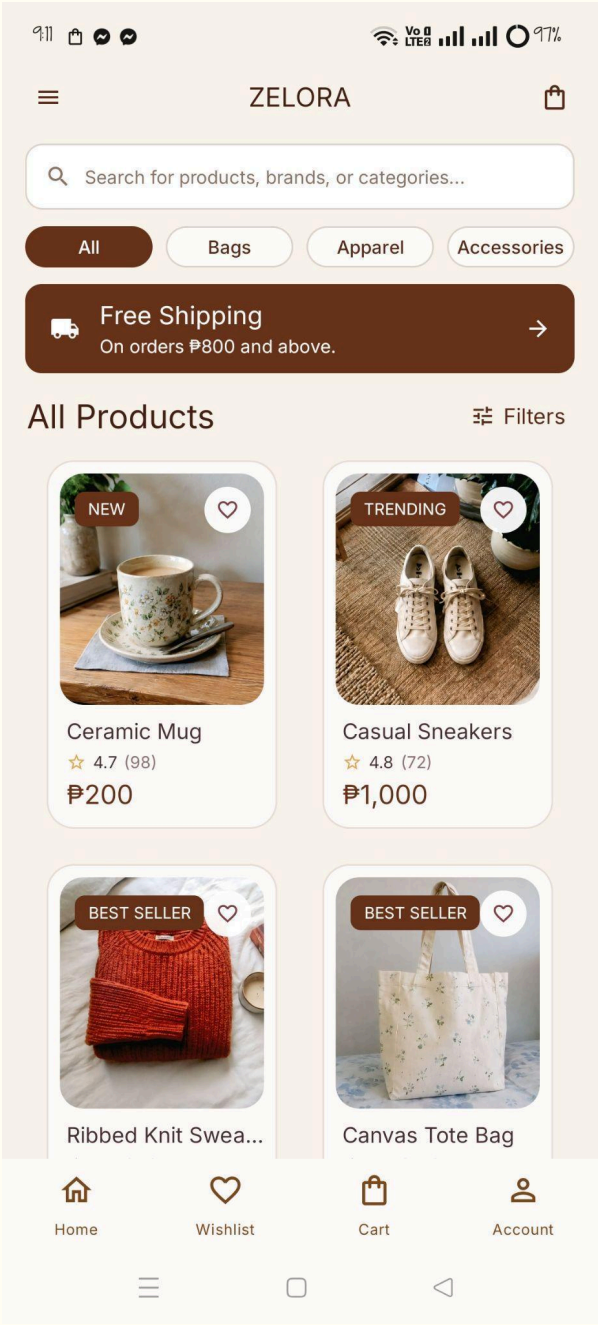
Future development should include systematic testing of registration, login, database persistence, product management, checkout, order status transitions, seller permissions, rider assignments, notifications, and administrative functions. Testing on both an Android emulator and physical devices can help identify compatibility issues and improve overall application reliability. By addressing these recommendations, future versions of Zelora can become more reliable, maintainable, secure, and suitable for a broader range of users and devices.

APPENDICES

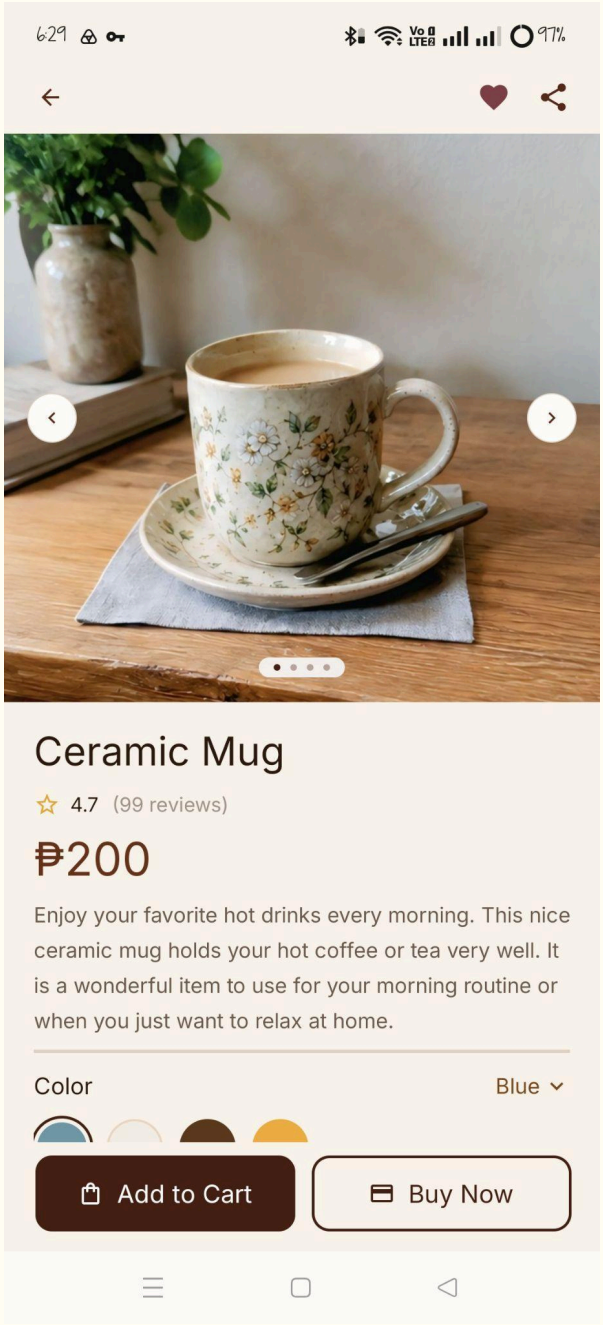
Appendix A. Final Application Screenshots



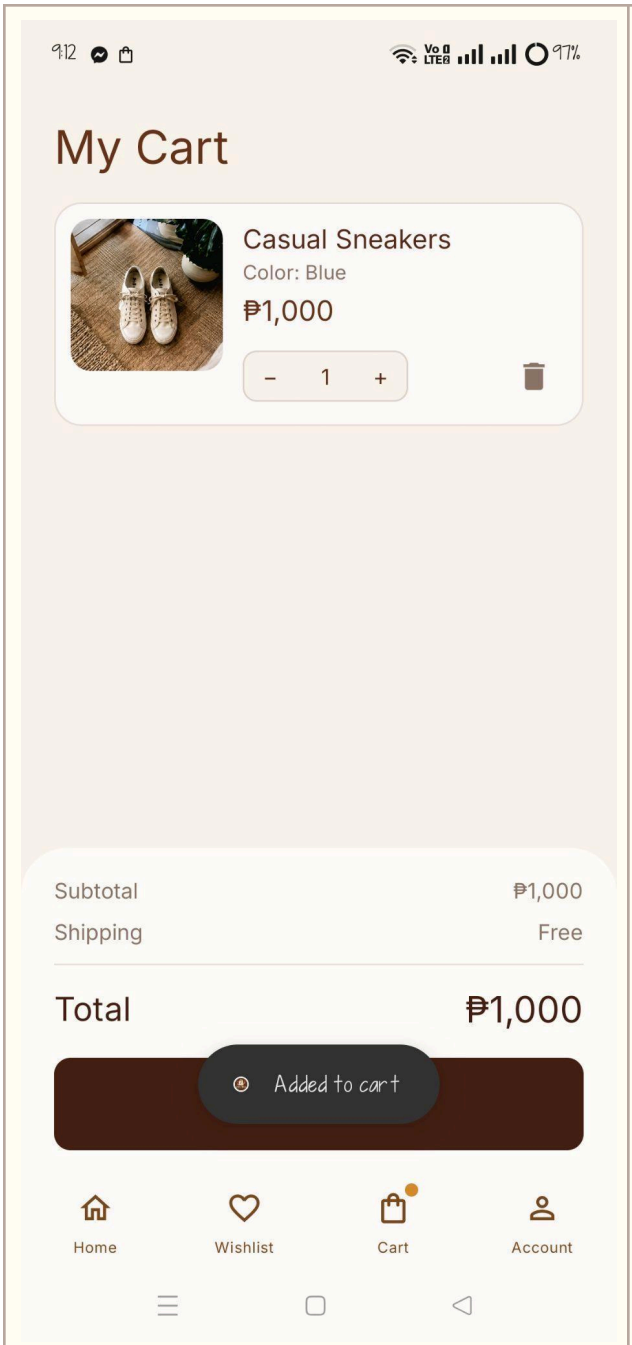
Login | Sign Up



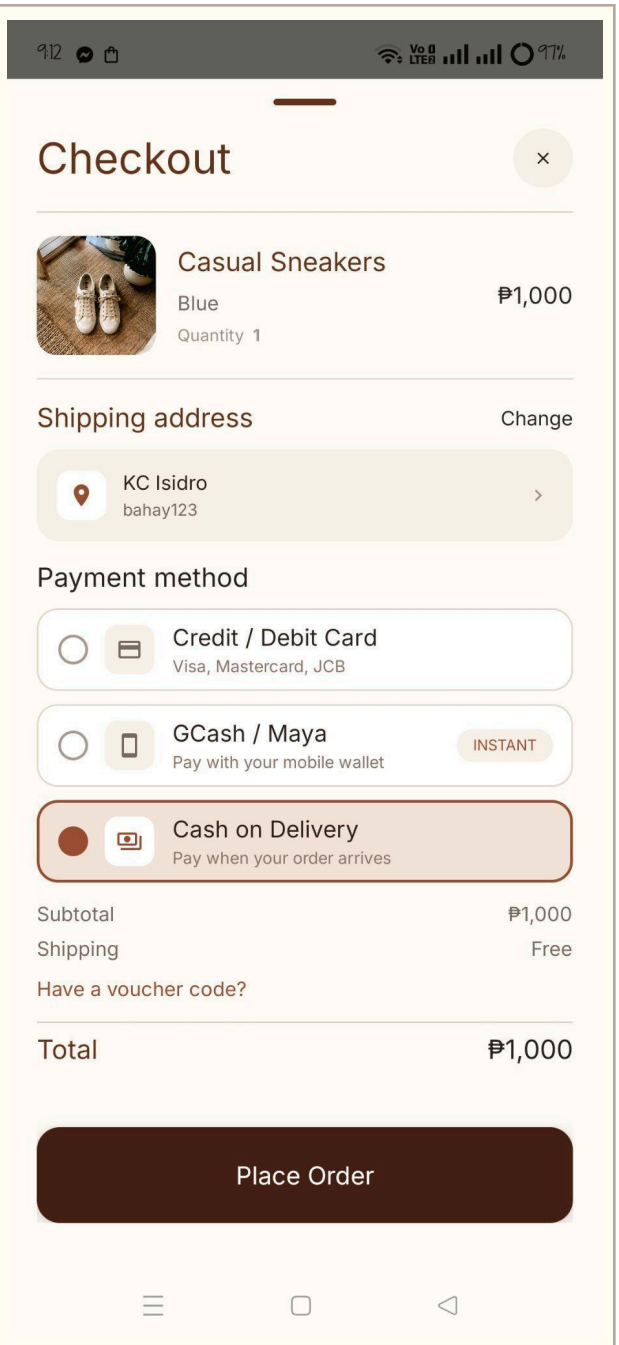
Home



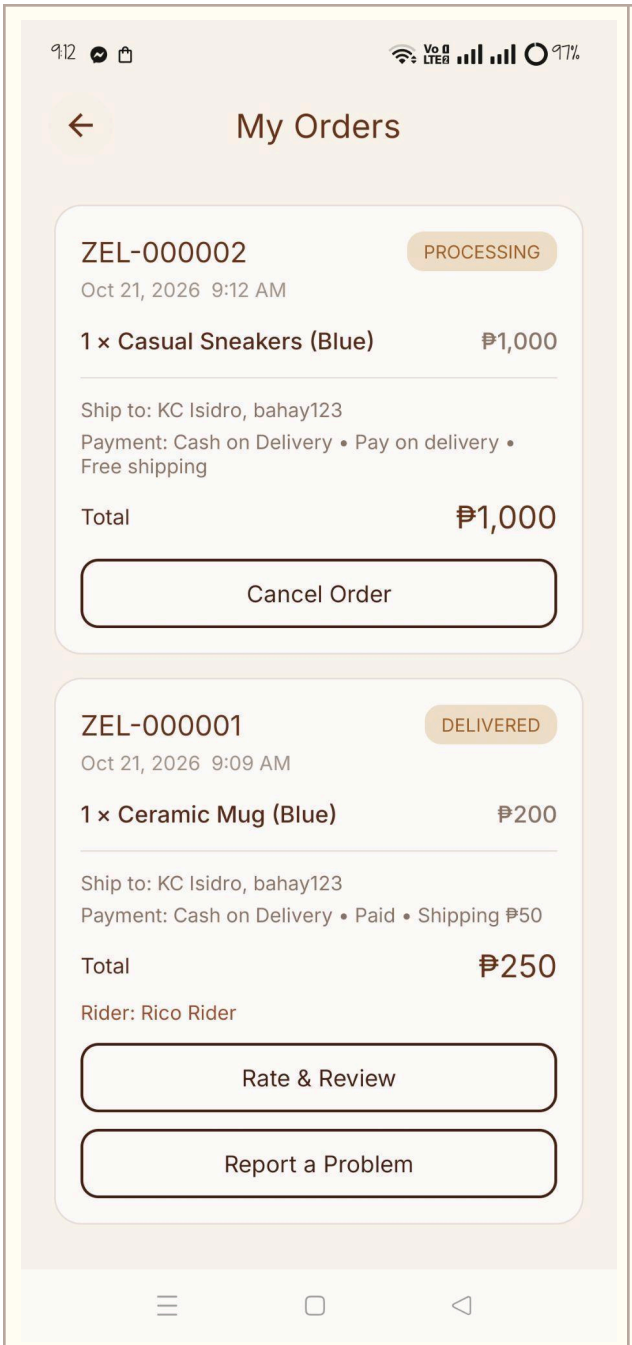
Product Detail



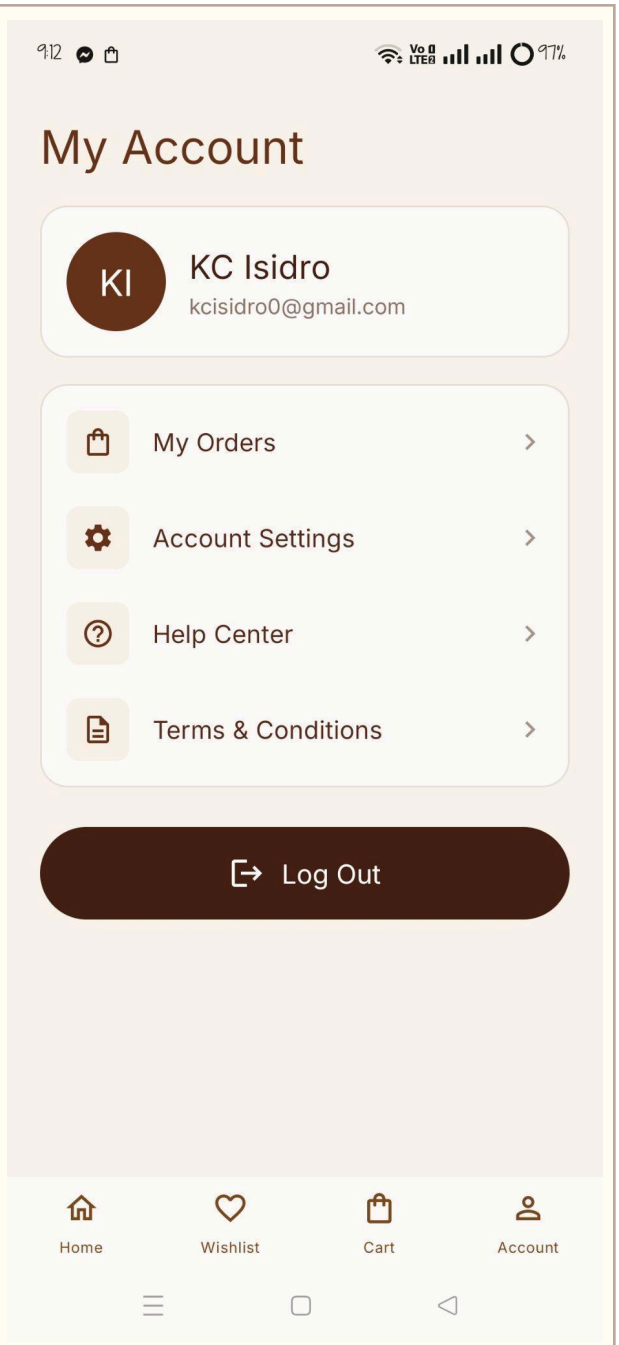
Cart



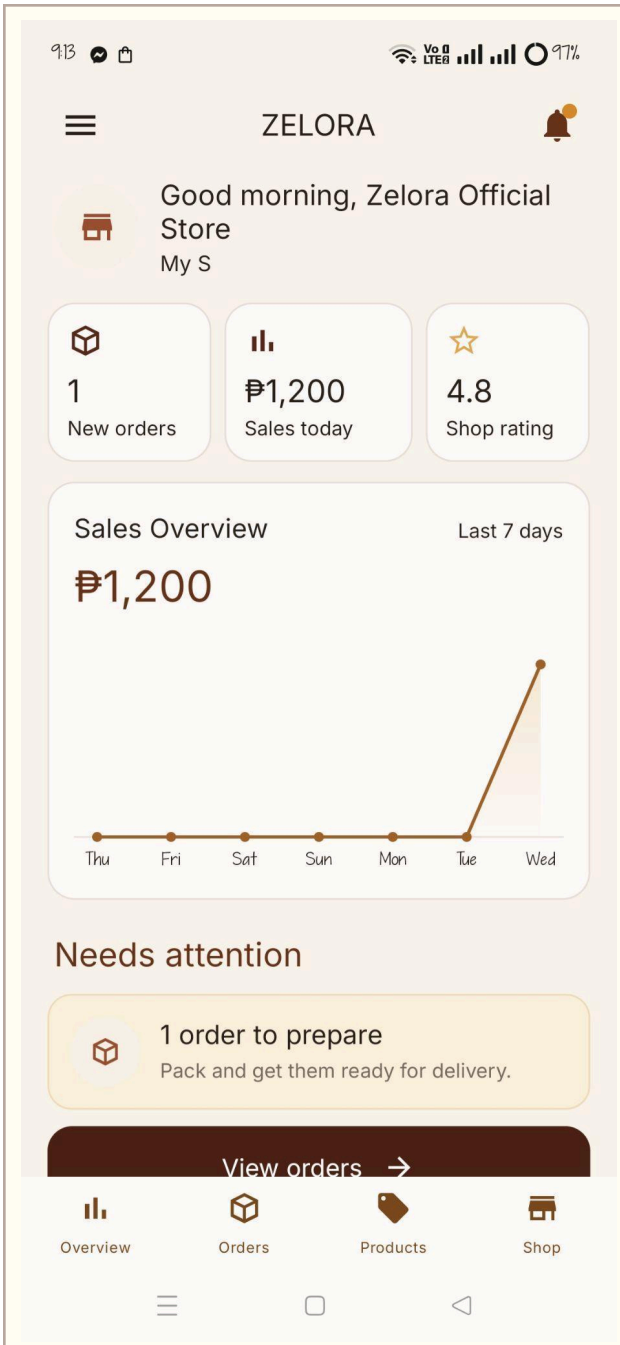
Checkout



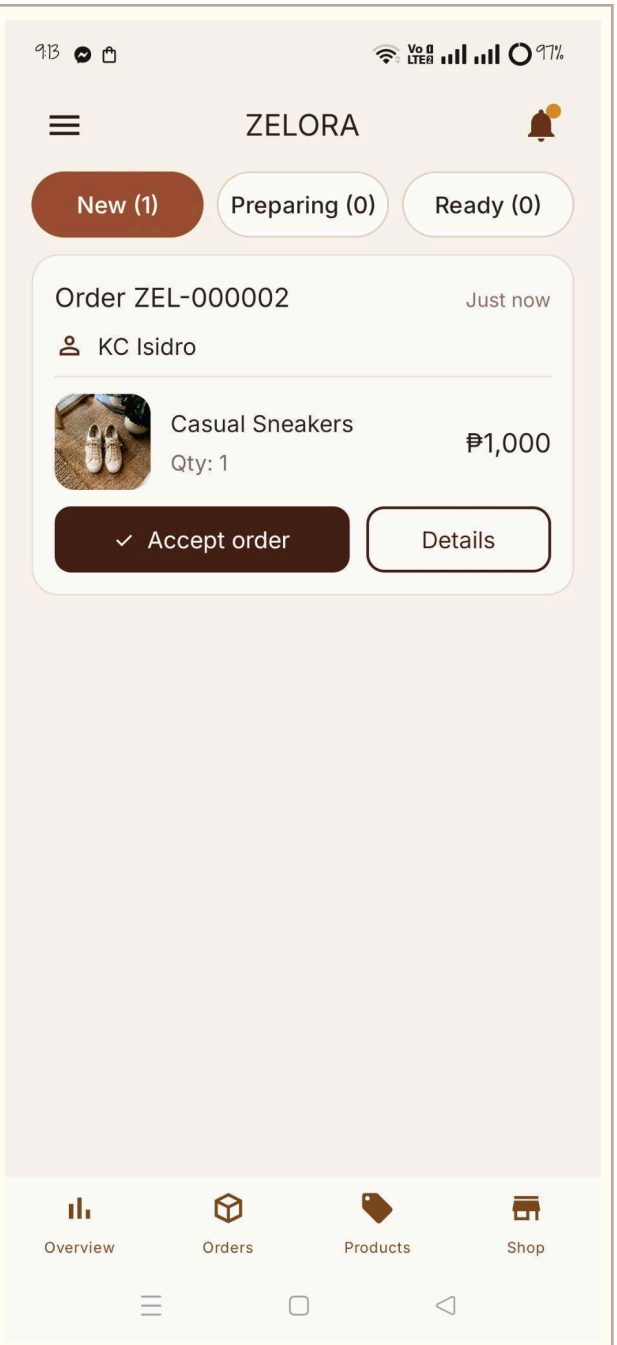
My Orders (with status)



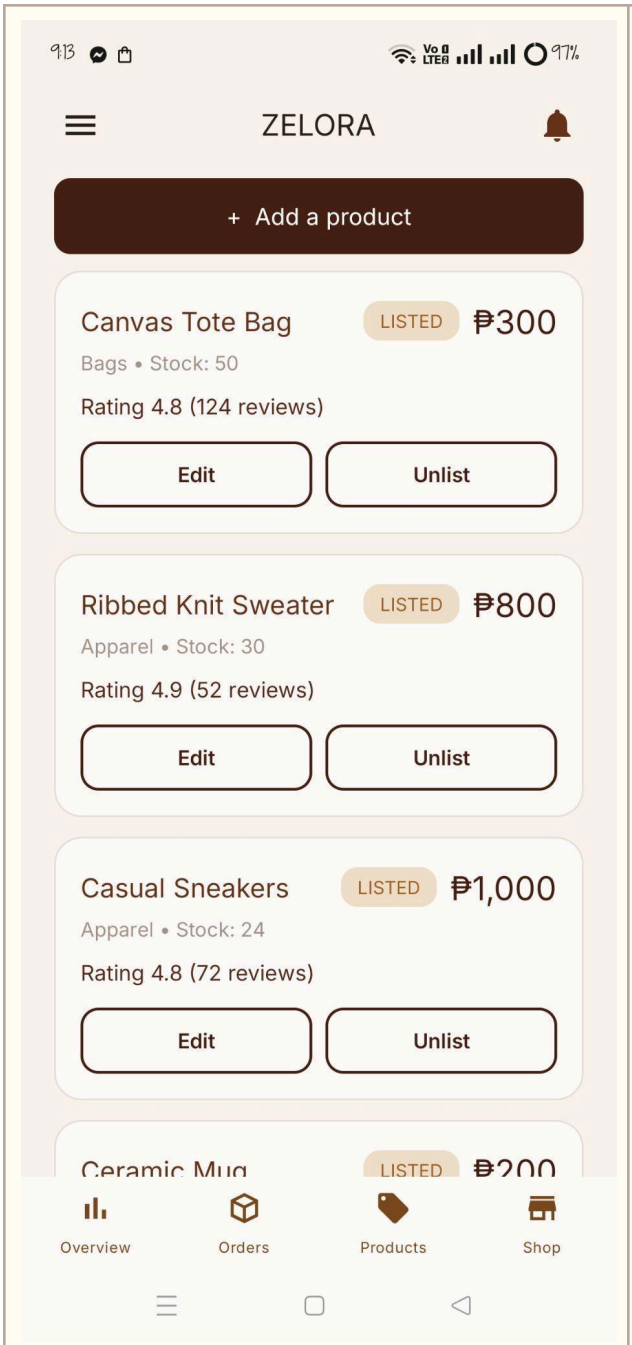
Account



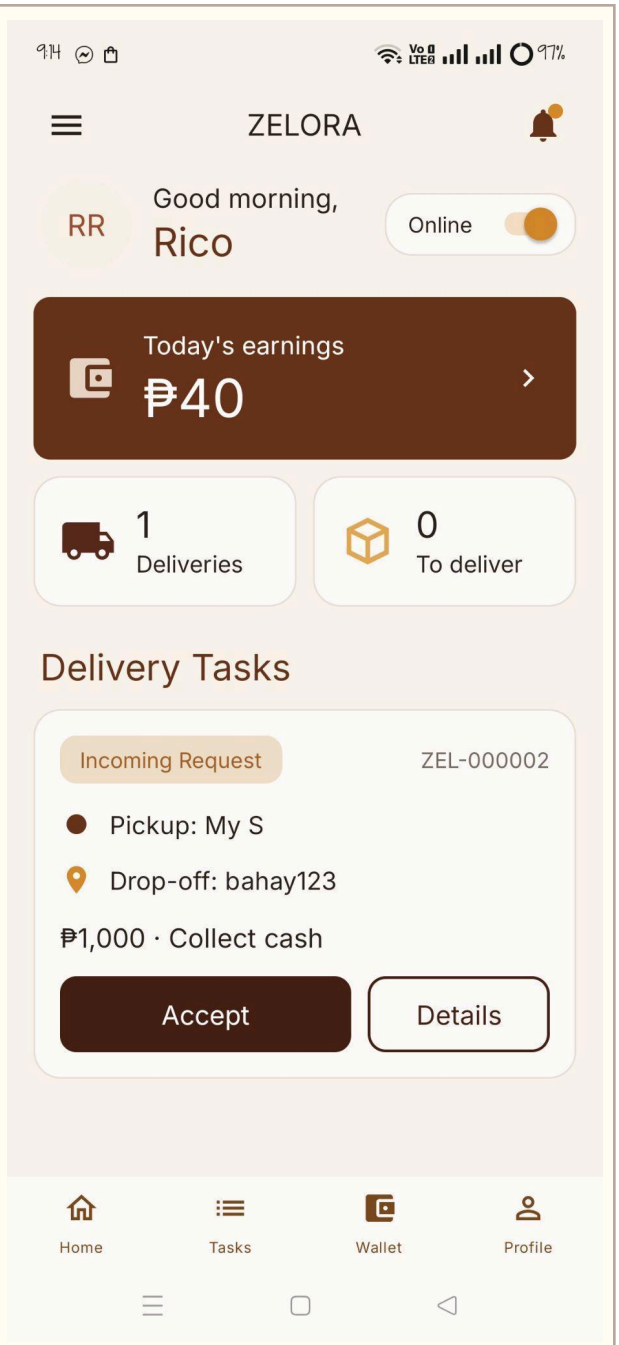
Seller Overview



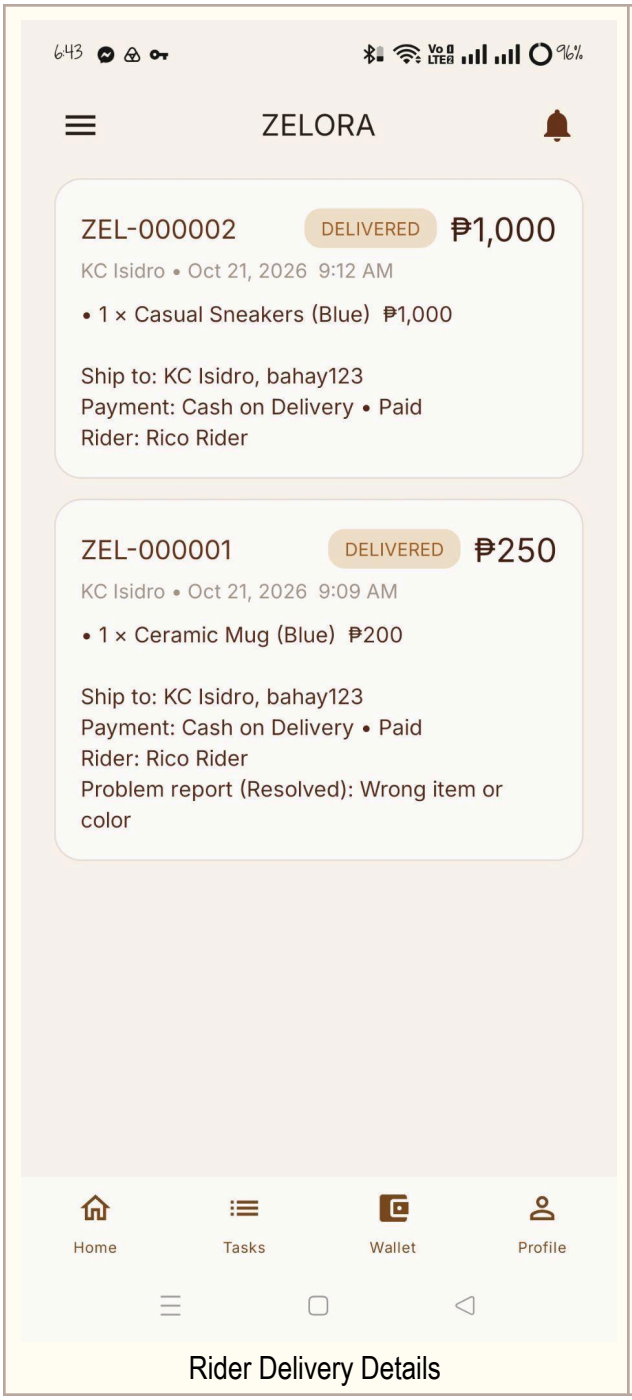
Seller Orders



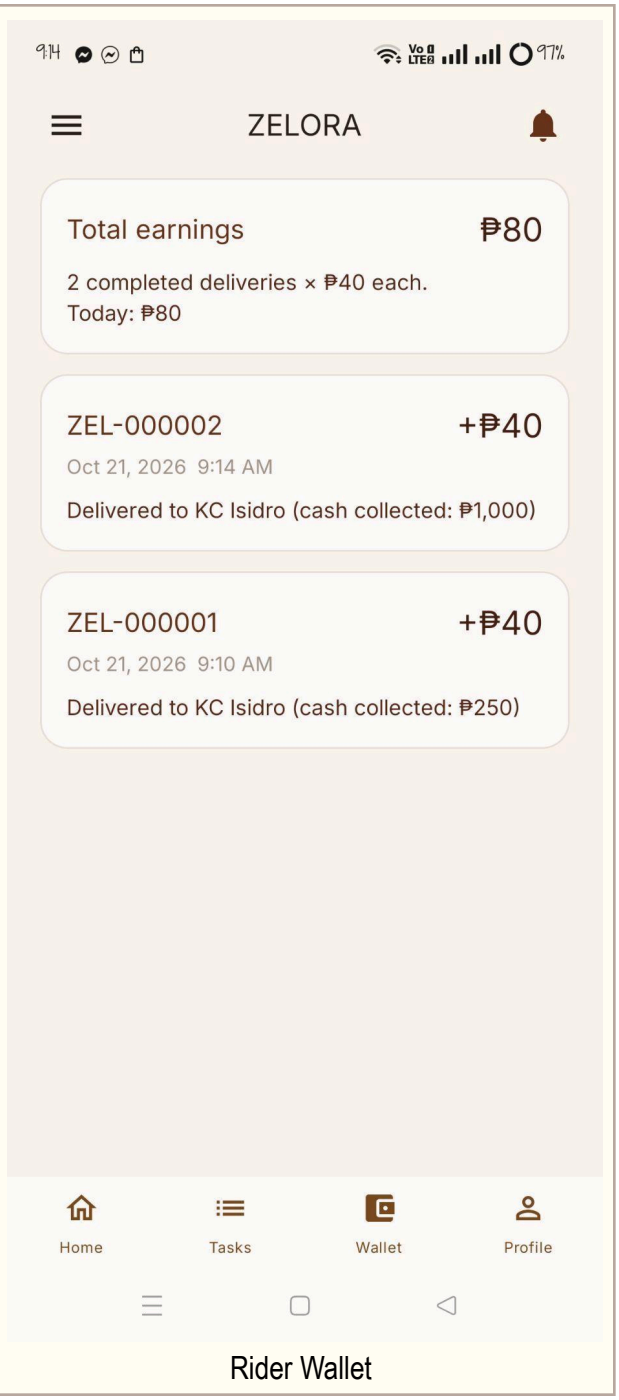
Seller Products



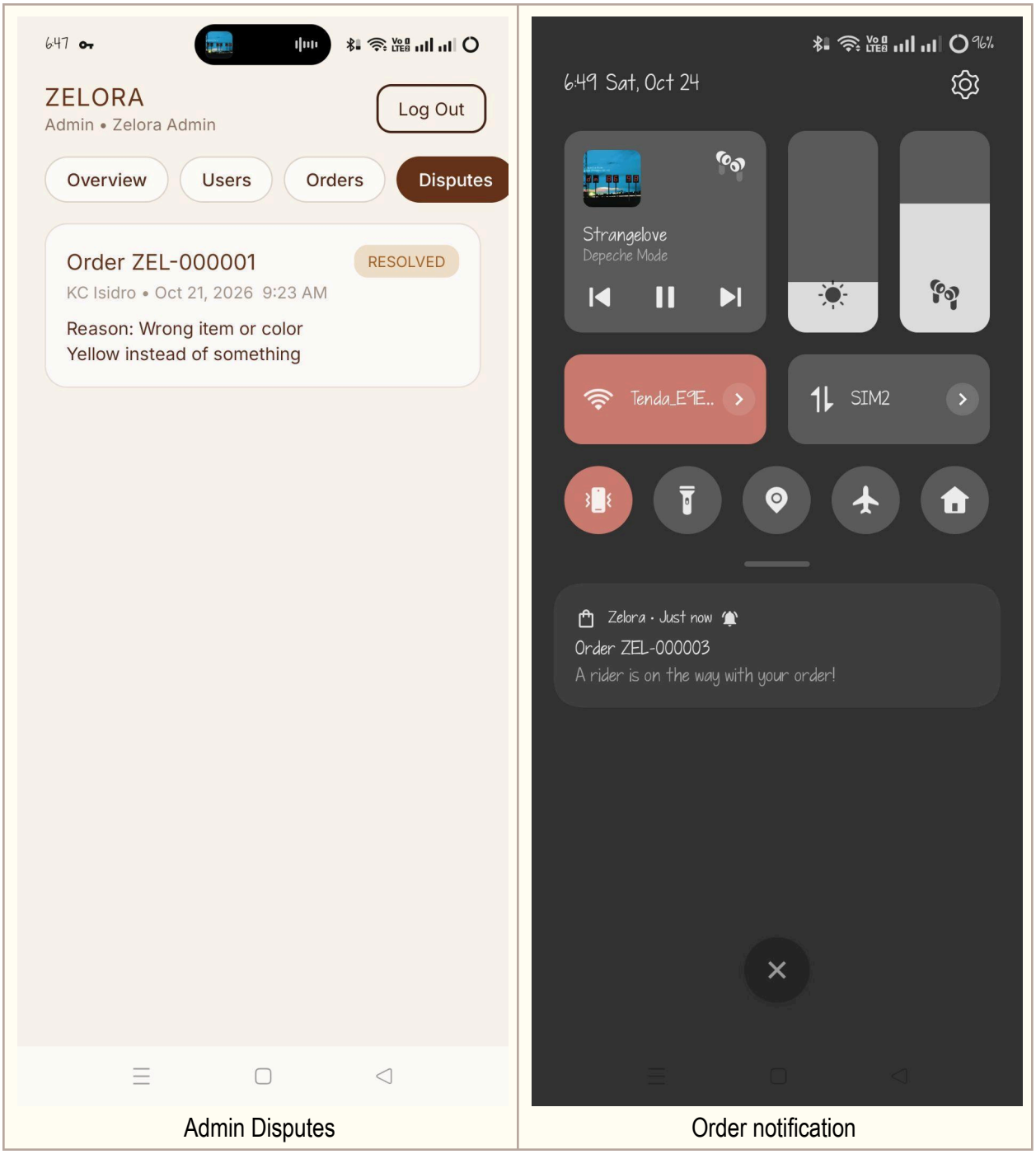
Rider Home



Rider Delivery Details



Rider Wallet



Admin Disputes

Order notification

Appendix B. GitHub Repository and Installation

Repository URL: [https://github.com/keshidesu/Zelora]

The repository contains the source code, README.md (description, features, technologies, installation, screenshots, developers and version), the design mockups, and the system design diagrams.

Installation. Open the Zelora folder in Android Studio, wait for Gradle Sync, choose an emulator (API 26 or newer) or a phone with USB debugging, and press Run. To build an APK: Build → Build Bundle(s)/APK(s) → Build APK(s).

Demo accounts. Customers sign up in the app. Staff accounts are created automatically on first launch:

Role	Email	Password
Seller	seller@zelora.com	Seller1234
Delivery Rider	rider@zelora.com	Rider1234
Admin	admin@zelora.com	Admin1234

Vouchers to try: WELCOME10 (10% off), SAVE100 (₱100 off orders of ₱1,000 or more) and FREESHIP (free shipping).

Appendix C. Acknowledgments and Third-Party Resources

- Fonts: Inter (Rasmus Andersson) and Bricolage Grotesque (Mathieu Triay), SIL Open Font License.
- Libraries: AndroidX (AppCompat, RecyclerView, ViewPager2, Fragment) and Material Components for Android, Apache License 2.0.
- Product photos, logo and mockups: the project author.
- Diagrams were produced with Graphviz and draw.io.

Use of AI tools. As allowed by the project instructions, an AI assistant (Claude, by Anthropic) was used as a development and learning resource to help write, review and test parts of the code and documentation. The author reviewed the code and is able to explain how it works.